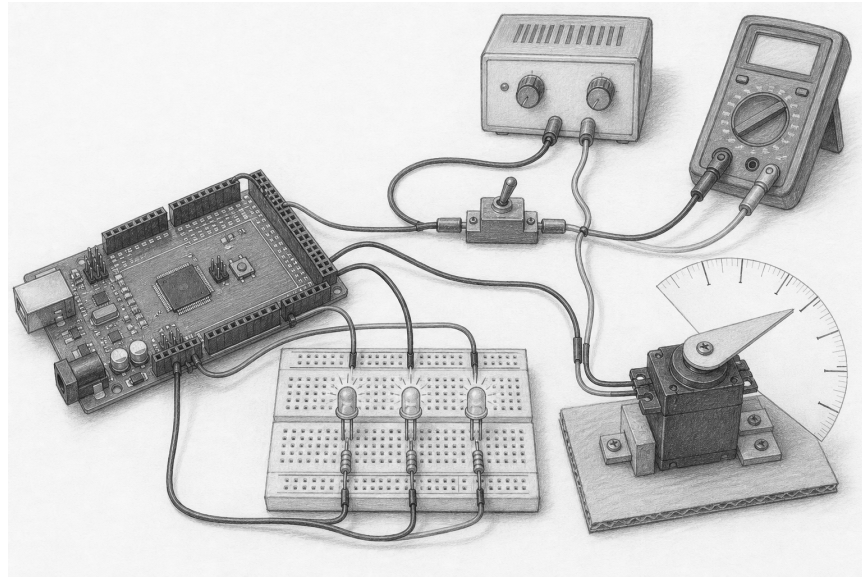


Separate command from motion

Lesson 017: bounded servo intent

Admit → command → measure → energize → observe



Orientation plate alternative text. Pencil view of a Mega, three resistor-limited command-marker LEDs, a separately powered servo carrying a paper pointer over a card, a physical supply switch, and meter probes at the load supply. Use the schematic and table below for exact connections.

Mission	Issue one bounded command while keeping logic intent, signal, external power, and physical position independently observable
Time	120–180 minutes plus a separate reviewed bench session
Prerequisites	Lessons 001–016; current measurement; timer/resource ownership
Energy class	E2: one restrained, externally powered micro servo with no load
Status	Host verified and Mega compiled; physical acceptance open
Primary evidence	D13, D30–D32, TP-S/D44, TP-V, supply current, paper pointer

1. Admit the exact physical system

This lesson does not admit a generic “SG90-compatible” servo. Before selecting a supply or pulse range, record the exact unit:

Marking

Primary source

Voltage range

Idle/running current

Supply model

Manufacturer

Pin order

Pulse range

Brief start/stall limit

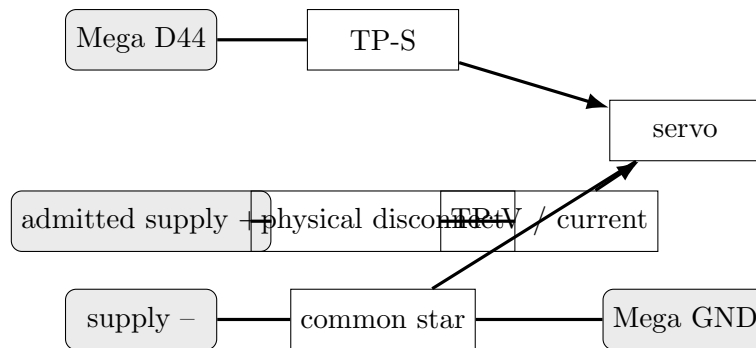
Current limit

The first fixture carries only a soft paper pointer over a flat, taped-down card. It has no linkage, latch, hard stop, spring, useful load, or pinch point. The permitted software range is narrower than observed mechanical travel. Never discover a pulse endpoint by driving into a stop.

Stop immediately for unexpected or reversed motion, chatter, stall, heat, odor, current-limit operation, supply sag, reset, lost common ground, malformed TP-S waveform, contact with the marked envelope, or disagreement between command marker and pointer. Open the physical servo-positive switch; do not investigate with load power present.

2. Keep the power domains explicit

Servo positive comes from a separately regulated, current-limited supply through a reachable physical switch or removable connector. It never connects to Mega 5 V, VIN, USB 5 V, or an I/O pin. Join supply negative and Mega GND at one named star point. Common ground joins signal references; it does not join positive rails.

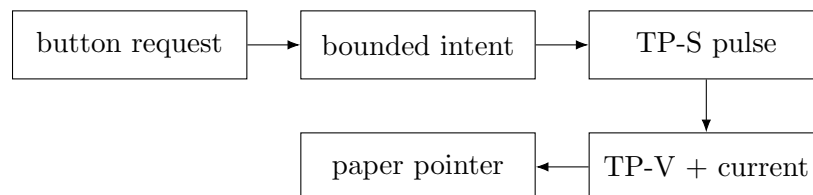


connect common ground first, signal second,
servo positive last; remove servo positive first

Evidence	Connection	Inactive state	Meaning
LogicReady	D13 built-in LED	off	logic resources acquired only
Low/Middle/High	D30/D31/D32 through 220 Ω LEDs	all off	accepted command range
TP-S	D44/OC5C relative to Mega GND	high impedance	pulse command
TP-V	servo positive to supply negative	switch open	load voltage
Current point	series in servo-positive lead	zero with switch open	load-domain energy
Pointer	soft paper on horn	unobstructed	observed shaft position

Attach meter and probe clips while all power is off. Put the current meter in series only through the documented current point and with the correct fused input/range. Never place a current-mode meter directly across TP-V.

3. Separate the claims



- A lit marker proves the program accepted a command; it does not prove a pulse or motion.
- TP-S proves an electrical waveform; it does not prove load power.
- TP-V and current prove external energy; they do not prove position.
- The pointer shows position but supplies no controller feedback.

“Commanded middle” and “observed at the middle mark” are separate record fields. A hobby servo normally cannot report actual shaft position. Detaching the pulse does not mechanically lock the horn. Holding a position may draw current and is not de-energized.

4. Read the pure bounded model

The pure model owns no pin, timer, or clock. Its position domain is 0–1000 permille; the reviewed example maps it to 1000–2000 microseconds and selects 500 as the safe position.

```
const adk::BoundedServoConfig servoConfig =
{
    {0, 1000, 1000, 2000},
    500
};

adk::BoundedServo servo (servoConfig);
```

`initialize()` validates the calibration and safe position but leaves the intent `Inactive`. `command(position)` produces a `Position` intent or rejects the position without changing the previous accepted intent, position, or pulse. `snapshot()` returns complete status, intent, position, and microseconds. `shutdown()` returns to `Inactive`, `NotInitialized`, safe position, and zero pulse.

The integer interpolation is deterministic:

$$p = p_{\min} + \frac{(x - x_{\min})(p_{\max} - p_{\min})}{(x_{\max} - x_{\min})}.$$

The first-class record requires increasing position and pulse endpoints. The admitted lesson record must also agree with the pointer’s named direction.

5. Validate configuration without pretending storage is solved

`ServoConfigurationRecord` has a 20-byte versioned encoding in 24 bytes of fixed capacity. It includes calibration, safe position, generation, magic, version, length, and CRC. It rejects erased, truncated, future-version, checksum-failed, and semantically invalid input. It does not silently clamp, repair, or widen a configuration.

```
adk::ServoConfigurationRecord record;

adk::Status status = record.save (configuration);
if (status.ok ())
{
    adk::Result<adk::ServoConfiguration> loaded = record.load ();
}
```

An empty record is `NotInitialized`; an unknown version is `Unsupported`; corruption is `HardwareFailure`. The canonical bench sketch encodes its reviewed compiled configuration, then decodes and compares the complete result before acquiring any hardware resource. This in-memory exercise defines portable bytes but does not claim EEPROM durability; storage transport and torn-write recovery belong to the later storage lesson.

6. Own D44 and Timer5

`MegaTimer5ServoPulseIo` is a first-class direct AVR adapter. It does not hide Arduino's global `Servo` library behind the registry.

```
adk::ExternalPowerDomainGate powerGate;
adk::MegaTimer5ServoPulseIo pulseIo;
adk::ServoOutput              pulseOutput (
    runtime.resources (),
    pulseIo,
    powerGate,
    adk::MegaTimer5ServoPulseIo::signalPin,
    1000,
    2000,
    adk::MegaTimer5ServoPulseIo::timer);
```

Initialization transactionally claims D44 and Timer5, then attaches an inert endpoint. The adapter uses a 20 ms frame and writes the bounded high time to OC5C. Timer5 ownership conflicts with PWM on D44, D45, and D46.

`ExternalPowerDomainGate::admit()` permits a command; it does not sense or switch external voltage. That distinction lets the learner verify TP-S while the physical servo-positive switch remains open.

Any compare-register or output-connect failure completely disables Timer5, drives D44 low, makes D44 high impedance, and leaves the endpoint detached. Further commands report `NotInitialized` until an explicit shutdown/reinitialize cycle.

7. Read the canonical run loop

The example's button advances center, low, center, and high. Its high-level flow appears before mechanics:

```
const adk::TimePoint now (millis ());

if (!commandRequested (now))
{
    return;
}

const uint16_t position = positions[nextPosition];

if (!commandPosition (position))
{
    stopLogic ();
    return;
}
```

`commandPosition()` admits the command path, asks `BoundedServo` for a pulse, sends it through `ServoOutput`, then lights exactly one independent command marker. A fault revokes command admission and stops the logic outputs. `stopLogic()` is intentionally not named “emergency stop”: the operator must still open physical servo power.

8. Stage the first run

1. With every source off, identify the servo pins from its primary documentation. Mount the servo and paper pointer; mark a keep-clear arc.

2. Connect common ground first and D44 second. Leave servo positive physically open. Inspect every net and probe clip.
3. Upload and reset with servo power open. The sketch first validates its record, acquires D44 and Timer5, then starts the 1500 μ s safe waveform and middle marker.
4. D13 on supports completion of that startup sequence. Observe the TP-S high time and 20 ms frame before pressing the button and before adding load power.
5. Set the exact documented voltage and current limit. Clear the arc, then deliberately close the physical load-power switch.
6. Record TP-V, current, middle marker, and pointer. Press through low, middle, and high while remaining outside the arc.
7. Open load power first and verify zero current. Only then stop the logic and verify TP-S absent and D44 high impedance.

Check — before closing servo power

- ☐ exact servo admitted ☐ supply voltage/current limit recorded ☐ pointer arc clear ☐ D13 on
- ☐ TP-S bounded ☐ physical disconnect reachable

9. CLI-only workflow

```
make host-test
make arduino-Lesson017BoundedServo
make size-check
make upload EXAMPLE=Lesson017BoundedServo PORT=/dev/ttyACM0
make lessons
make lessons-check
make site
make site-check
```

Use **make boards** to find the port. Open load power before upload, reset, or reconnect. The sketch emits no Serial data. Optional **make monitor** and **make serial-log** records cannot replace the marker, TP-S, TP-V, current, and pointer evidence.

The current Mega build measures 6656 bytes of flash and 323 bytes of static RAM. Toolchain compilation is not hardware verification.

10. Diagnose from left to right

Observation	Next check	Separation
D13 off	keep load power open; inspect resource/status trace	initialization from physical wiring
Marker on, no TP-S	keep load power open; inspect command admission and Timer5 conflict	intent from pulse
TP-S valid, TP-V zero	inspect physical disconnect and admitted supply	logic waveform from load power
TP-V sags/current limits	open power immediately; verify exact servo, supply, polarity, and obstruction	energy source from software intent
Marker and TP-S agree, pointer differs	open power; inspect horn attachment, direction, and admitted calibration	command from physical outcome
Chatter or heat	open power immediately; do not retry in software	unsafe physical state from recoverable logic error

11. Deterministic proof

Host tests enumerate calibration endpoints and overflow-safe interpolation; invalid safe positions; record corruption at every byte; D44 and Timer5 conflicts; claim rollback; admission closed; pulse boundaries; compare and connect failures; fail-closed teardown; repeated lifecycle calls; destruction; and resource reuse. Injected Timer5 registers record exact ordered operations.

```
make host-test
make host-test-exceptions
make quality
```

These tests prove deterministic software behavior. They cannot identify a servo, set a supply limit, measure a pulse, or accept a physical fixture.

12. Bench acceptance record

Board/revision

Servo identity

Supply/current limit

Meter/probe

Sketch revision

Primary source

Physical disconnect

Reviewer/date

Claim	Prediction	Observation	Interpretation
Logic acquired	D13 on; load power open		
Center command	middle marker; TP-S 1500 μ s		
Low/middle/high	marker, pulse, pointer agree separately		
Load power	TP-V in range; current below admitted limit		
Reset/upload	load power remains phys- ically open		
Shutdown	load current zero first; TP-S absent; D44 high- Z		
No back-power	asymmetric power checks meet procedure		

Acceptance status: ☐ not run ☐ pass ☐ deviation attached

Exercises and next composition

1. For commands 0, 250, 500, 750, and 1000, calculate the expected pulse. Compare arithmetic with host snapshots before touching hardware.
2. Corrupt each field of an exported configuration record. Predict the exact status and prove that no pulse intent results.
3. Claim Timer5 or D45 PWM first. Predict the resource error and the unchanged TP-S state.
4. Design a fourth evidence channel for external power without allowing software to switch that rail. State what it proves and what it cannot.
5. Carry the pure bounded intent into lesson 018's inert access trainer. Preserve "close requested" separately from "pointer observed closed."

Use the HTML page for maintained API links, errata, source, and copyable commands. Use this PDF for the staged experiment, worksheets, and signed bench record. Lesson 018 adds a keypad and display to an inert paper-pointer trainer; it is not a lock or security product.

Primary references

- Arduino Mega 2560 Rev3 documentation and pinout
- Microchip ATmega2560 Timer5 and electrical documentation
- Arduino Servo library guidance for external power and common ground
- TowerPro SG90 manufacturer page—identity example, not clone authority

Deferred evidence. No physical voltage, current, waveform, travel, reset, shutdown, or power-removal claim is accepted until a person completes this record for the exact named fixture.